SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	P.Akhila	**Reg. No.:**	192425225
Date:	24-10-2026	**Duration:**	60 minutes

Code of Conduct

I **Akhila Pochimireddy(192425225)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Akhila.P

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.

- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.

Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Annexure A – Case Study Analysis
Smart Agricultural Marketplace Platform

1. Understand the Problem Statement

1.1 The Problem in My Own Words

Small and medium farmers usually cannot sell directly to the people who need their produce. Between the farm and the final buyer sit several intermediaries (commission agents, traders, and transporters). Each one adds a margin and a delay. As a result, the farmer rarely knows the real market demand or the fair price, while wholesalers, retailers, and consumers have no reliable way to see what is available, from where, and at what quality. Communication happens through phone calls and informal arrangements, so orders, payments, and deliveries are hard to track and disputes are hard to settle.

The task is to design a **Smart Agricultural Marketplace Platform**: a single online system where farmers list their produce, buyers (wholesalers, retailers, and consumers) discover and order it, payments are made securely, deliveries are tracked, and everyone can see transaction history, prices, and ratings.

1.2 Objectives of the Proposed System

- Register and verify farmers and buyers (wholesalers, retailers, consumers).
- Let farmers create and manage product listings (crop, variety, grade, quantity, price, location, harvest date).
- Display real-time product availability and prevent over-selling of stock.
- Manage the complete order lifecycle: placement, confirmation, packing, dispatch, delivery, and closure.
- Support secure online payments with proper receipts and refunds.
- Track deliveries and give live status updates to both parties.
- Maintain a permanent transaction history for audit and dispute resolution.
- Provide market analytics (price trends, demand by crop and region, supply forecasts).
- Manage ratings and feedback to build trust between participants.

1.3 Scope of the Problem

Aspect	Details
In scope	Web portal and mobile app for all user roles; user registration and verification; product listing and search; cart and order management; payment gateway integration; delivery tracking; transaction ledger; analytics dashboards; ratings and reviews; admin panel; notifications (SMS / e-mail / push); multilingual interface.
Out of scope (initial release)	Own fleet of vehicles, cold-storage warehouses, crop insurance, farm loans, export documentation, and IoT sensor integration on the farm.
Users	Farmers, wholesalers, retailers, consumers, delivery partners, and platform administrators.
Geography	Pilot in one state (for example Tamil Nadu), then expansion region by region.

2. Identify Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role in the system	Responsibilities
Farmers	Sellers and producers of agricultural goods.	Register and verify identity; list produce with accurate quantity, grade, and price; confirm orders; pack and hand over goods; respond to feedback.
Wholesalers	Bulk buyers who supply markets and processors.	Place bulk or contract orders; make payments on time; confirm receipt; rate farmers.
Retailers	Shops, supermarkets, and restaurants buying in medium quantity.	Browse listings, order regularly, check quality on delivery, give feedback.
Consumers	End buyers purchasing small quantities.	Order produce, pay securely, confirm delivery, rate products and sellers.
Delivery / logistics partners	Transport the goods from farm or collection point to the buyer.	Accept pickup tasks, update delivery status, ensure safe and timely handover.
Payment gateway / bank	External financial service.	Process payments, hold money in escrow, settle to farmers, handle refunds.
Platform administrator	Operates and governs the marketplace.	Verify users, approve listings, resolve disputes, manage categories, monitor fraud, and publish reports.
Government / agriculture agencies	Regulators and information providers.	Provide MSP and mandi price references; enforce quality and tax rules (GST, licences).

Stakeholder	Role in the system	Responsibilities
Development and support team	Builds and maintains the platform.	Develop, test, deploy, monitor, and support users.

2.2 Major Challenges Faced by Stakeholders

Stakeholder	Challenges
Farmers	Limited market access; dependence on middlemen who reduce their share of the price; no price transparency; delayed or uncertain payment; low digital literacy; poor internet in rural areas; perishable goods that lose value if not sold quickly.
Wholesalers / Retailers	Difficulty finding reliable suppliers; inconsistent quality and quantity; no visibility of stock before travelling; lack of traceability; price fluctuation.
Consumers	Limited access to fresh produce at fair prices; no information on origin or quality; fear of online payment fraud.
Delivery partners	Rural pick-up locations, cold-chain needs, and time-sensitive delivery of perishables.
Administrator	Fake listings, fake users, payment fraud, disputes over quality, and keeping data accurate at scale.
All parties	Communication gaps, trust issues, and lack of proof of transactions.

3. Analyze the Current Approach

3.1 Existing System / Current Process

At present, most agricultural trade follows the traditional route: the farmer transports produce to the local mandi (regulated market) or sells to a village trader or commission agent. The agent auctions or negotiates with wholesalers, who then sell to retailers, and retailers finally sell to consumers. Prices are fixed by negotiation on the day, orders are agreed by phone or in person, payment is often in cash or delayed by days, and records are kept in paper registers. Some farmers use WhatsApp groups or generic e-commerce sites, but these are not designed for agriculture.

3.2 Strengths and Limitations

Strengths	Limitations
Familiar to farmers and traders; no technical skills needed.	Multiple intermediaries reduce the farmer’s income and raise the consumer’s price.
Physical inspection of goods before purchase.	No price transparency; prices differ from mandi to mandi.
Established trust through personal relationships.	Trust is limited to known contacts; new buyers and sellers are excluded.

Strengths	Limitations
Immediate cash settlement is possible.	Cash handling is unsafe; payment delays and non-payment are common.
Local mandis give assured demand for large volumes.	Travel cost and spoilage while waiting at the mandi; no delivery tracking.
	Paper records or none: no transaction history, no analytics, and disputes are hard to settle.

3.3 Gaps That Need to Be Addressed

- **Market access gap:** farmers cannot reach buyers outside their locality.
- **Information gap:** no common source for live price, demand, and stock information.
- **Trust and payment gap:** no secure payment, no ratings, and no verified identities.
- **Traceability gap:** no tracking of goods from farm to buyer, and no digital record of transactions.
- **Coordination gap:** supply and demand are not matched in advance, causing waste in surplus and shortage in other regions.
- **Decision-support gap:** no analytics to help farmers choose what to grow and when to sell.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Proposed Software-Based Solution

The proposed solution is a cloud-hosted, role-based **Smart Agricultural Marketplace Platform** with a web portal and a lightweight, multilingual mobile app (with low-bandwidth support for rural users). It uses a **layered architecture**:

- **Presentation layer:** React web app and mobile app, with role-based dashboards.
- **Application layer:** REST APIs exposing independent business modules (a modular monolith that can evolve into microservices as load grows).
- **Data layer:** relational database (PostgreSQL) for users, listings, orders, and payments; a cache (Redis) for availability and search; object storage for product images.
- **Integration layer:** payment gateway (UPI / cards / net banking), SMS and e-mail service, maps and GPS service, and a mandi price data feed.

Core workflow: Farmer lists product → Buyer searches and orders → Farmer confirms → Buyer pays (held in escrow) → Delivery partner picks up and delivers (status tracked) → Buyer confirms receipt → Payment released to farmer → Both parties rate each other → Transaction stored in history and analytics.

4.2 Major Functional Modules

Module	Description
1. User Management	Registration, login (OTP / password), role assignment, KYC verification (Aadhaar / GST / farmer ID), profile management.

Module	Description
2. Product Listing	Farmers add, edit, and remove listings with category, grade, quantity, price, images, harvest date, and pickup location.
3. Availability & Search	Real-time stock display, filters by crop, price, location, and quality, and automatic stock update after each order.
4. Order Management	Cart, bulk order and quotation request, order confirmation or rejection, cancellation, and order status workflow.
5. Payment	Secure online payment, escrow holding, invoicing, refunds, and settlement to the farmer's bank account.
6. Delivery Tracking	Assign delivery partner, GPS-based live status (Packed, Picked up, In transit, Delivered), proof of delivery.
7. Transaction History	Immutable ledger of orders and payments, downloadable statements and invoices.
8. Market Analytics	Price trends, demand heat-maps, best-selling crops, and seasonal forecasts through dashboards.
9. Ratings & Feedback	Two-way ratings, written reviews, complaint registration, and dispute handling.
10. Notification & Admin	SMS, e-mail, and push alerts; admin tools for verification, moderation, reports, and system configuration.

4.3 Functional Requirements

ID	Requirement
FR1	The system shall allow farmers and buyers to register, verify identity, log in, and manage their profiles.
FR2	The system shall allow farmers to create, update, and delete product listings with images and prices.
FR3	The system shall display product availability in real time and reduce stock automatically when an order is confirmed.
FR4	The system shall allow buyers to search, filter, and compare products and place single or bulk orders.
FR5	The system shall allow farmers to accept or reject orders, and buyers to cancel orders under defined rules.
FR6	The system shall support secure online payment, escrow, refunds, and invoice generation.
FR7	The system shall track delivery status and notify buyers and farmers at each stage.
FR8	The system shall store and display complete transaction histories for every user.

ID	Requirement
FR9	The system shall provide market analytics, including price trends and demand by crop and region.
FR10	The system shall allow users to rate and review each other and to raise complaints.
FR11	The system shall allow administrators to verify users, moderate listings, and resolve disputes.

4.4 Non-Functional Requirements

Category	Requirement
Performance	Pages load in under 3 seconds on 3G; search results in under 2 seconds; support 10,000 concurrent users.
Security	HTTPS/TLS, password hashing, OTP, role-based access control, PCI-DSS compliant payment gateway, and protection against SQL injection and XSS.
Reliability / Availability	99.5% uptime, automated daily backups, and failover for the database.
Scalability	Horizontal scaling of application servers; modules deployable separately.
Usability	Simple icon-based UI, regional languages, voice support, and low-bandwidth mode.
Maintainability	Clean code, documentation, automated tests (minimum 80% coverage), and version control.
Portability	Works on Android, iOS, and major browsers.
Data integrity & Compliance	Transactions are ACID-compliant and logged for audit; follows IT Act and data-privacy norms.

4.5 Software Engineering Principles Incorporated

Principle	How it is incorporated
Modularity	The system is split into ten loosely coupled modules with clear APIs, so each can be developed, tested, and changed independently.
Scalability	Stateless services behind a load balancer, caching, database indexing, and cloud auto-scaling to handle harvest-season peaks.
Maintainability	Layered design, coding standards, code reviews, Git version control, CI/CD pipeline, and complete documentation make changes easy and low-risk.
Reliability	Input validation, exception handling, transaction rollback, health monitoring, backups, and automated regression tests.
Usability	User-centred design with prototypes tested with real farmers; local languages; minimal steps to list a product or place an order.

Principle	How it is incorporated
Security	Authentication, authorization, encryption of data in transit and at rest, escrow payments, audit logs, and fraud detection rules.
Reusability & Abstraction	Common services (authentication, notification, payment) are reused by all modules; third-party services are accessed through interfaces so they can be replaced.

5. Justify the Chosen Methodology/Framework

5.1 Selected Model: Agile – Scrum

I have selected the **Agile methodology using the Scrum framework**, supported by **CI/CD (DevOps) practices** for build, test, and deployment. Work is organised into 2-week sprints with a Product Owner, Scrum Master, and a cross-functional development team.

Scrum element	Application to this project
Product Owner	Represents farmers, buyers, and the business; maintains the product backlog.
Sprint 1	User registration and verification; product listing.
Sprint 2	Search, availability, and order management (first working marketplace = MVP).
Sprint 3	Payment integration and escrow.
Sprint 4	Delivery tracking and notifications.
Sprint 5	Transaction history, ratings, and feedback.
Sprint 6	Market analytics, admin tools, performance and security testing, pilot release.
Ceremonies	Sprint planning, daily stand-up, sprint review with farmers and buyers, and retrospective.

5.2 Why Scrum Is Suitable

- **Changing requirements:** farmers' and buyers' needs are not fully known at the start and will change with seasons and market feedback. Scrum welcomes change at every sprint, whereas Waterfall does not.
- **Early value and feedback:** a working MVP (listing + ordering) is released in about 4 weeks, allowing real users to validate the idea before heavy investment.
- **Multiple stakeholders:** regular sprint reviews keep farmers, buyers, and administrators involved in decisions.
- **Risk control:** high-risk items such as payment and security are built and tested early and in small increments.
- **Modular product:** the module structure maps naturally to the product backlog and sprint goals.
- **Comparison:** Waterfall is too rigid and delivers late; Spiral is heavy for a project of this size; Scrum with DevOps balances flexibility, speed, and quality.

5.3 Support for Project Development and Maintenance

- Prioritised backlog and sprint goals give clear progress tracking (burn-down charts, velocity).
- Continuous integration, automated testing, and automated deployment reduce defects and release effort.
- Definition of Done and code reviews maintain consistent quality.
- After the launch, the same backlog and sprint cycle handle bug fixes, new features, and seasonal changes without a separate maintenance process.
- Retrospectives continuously improve the team's process.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits

- **Improved market access** for farmers, who can reach buyers beyond their village or mandi.
- **Better product visibility** through searchable, photo-based listings.
- **Simplified transactions** with one platform for ordering, payment, and delivery.
- **Improved price transparency** via live price and trend information.
- **Reduced communication gaps** through status notifications and in-app messaging.
- **Better supply-demand coordination** using demand analytics, so farmers plan sales and crops better and wastage reduces.
- Higher income for farmers, fairer prices for consumers, and trusted, auditable records for everyone.

6.2 How the Solution Addresses the Identified Challenges

Challenge	Solution feature
Dependence on middlemen	Direct farmer-to-buyer listing and ordering.
No price transparency	Market analytics and visible listing prices.
Payment delays and fraud	Escrow-based secure payment and automatic settlement.
Lack of trust	KYC verification, ratings, and feedback.
No delivery visibility	GPS-based delivery tracking with notifications.
No records or analytics	Transaction ledger and dashboards.
Low digital literacy	Regional languages, simple UI, and voice support.

6.3 Potential Risks, Implementation Challenges and Limitations

Risk / Limitation	Impact	Mitigation
Low adoption by farmers (digital literacy, trust)	Fewer listings, weak marketplace.	Training camps, farmer-producer organisations as local partners, simple UI.
Poor rural internet connectivity	Delayed updates or failed orders.	Low-bandwidth mode, SMS fallback, offline draft listings.

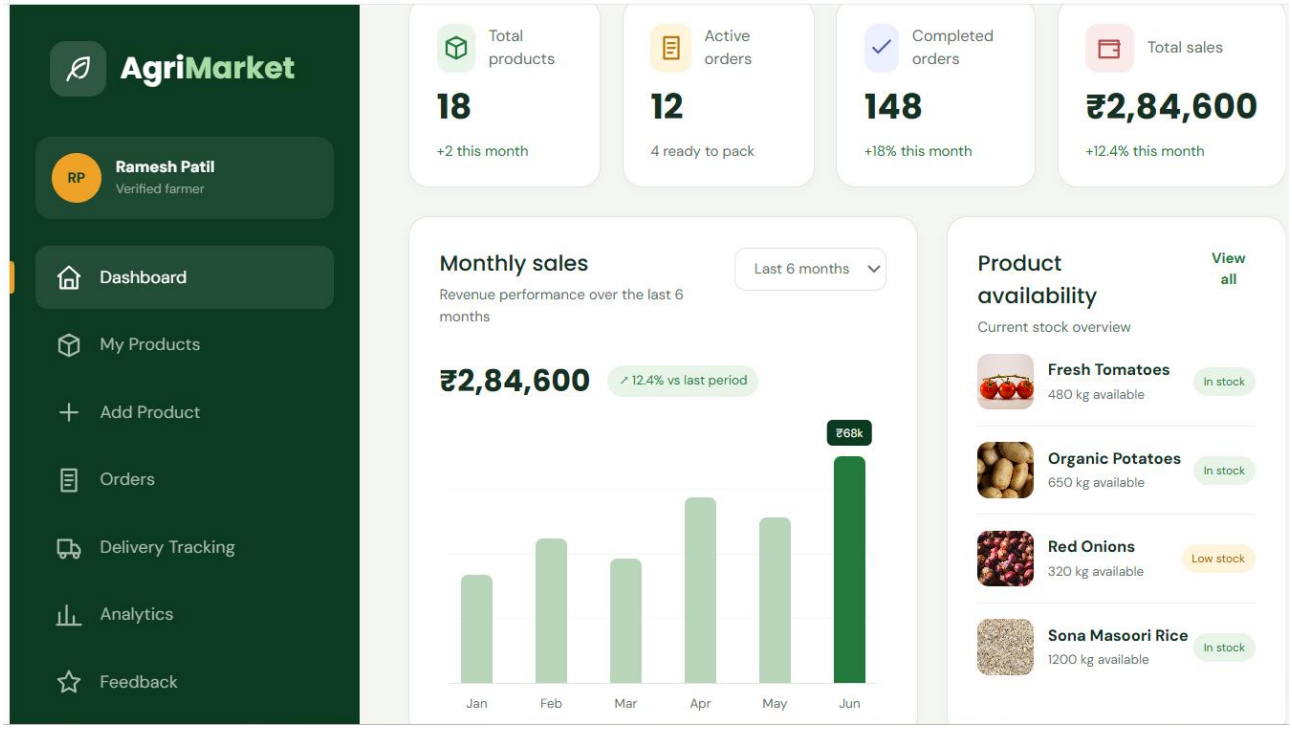
Risk / Limitation	Impact	Mitigation
Perishable goods and quality disputes	Refunds, rating damage.	Grade standards, photo evidence, escrow, dispute resolution.
Logistics and cold-chain constraints	Late delivery and spoilage.	Partner with local logistics providers; collection points.
Payment fraud and data breaches	Financial and reputation loss.	PCI-DSS gateway, encryption, audit logs, security testing.
Scope creep and schedule delay	Budget overrun.	Prioritised backlog, fixed sprint scope, MVP-first approach.
Dependence on third-party services	Outages affect payments or SMS.	Abstraction interfaces and alternate providers.
Limitations (first release)	No own fleet, no cold storage, and no export support.	Planned for later releases.

6.4 Future Enhancements

- AI-based price prediction and crop recommendation using historical and weather data.
- Blockchain-based traceability from farm to fork.
- IoT integration for soil, weather, and cold-storage monitoring.
- Voice assistant and chatbot support in more regional languages.
- Contract farming, forward booking, and group-buying for small farmers.
- Integration with crop insurance, agri-loans, and government subsidy schemes.
- Route optimisation for delivery and a shared-logistics model.
- Expansion to export markets and a farmer community forum.

Conclusion

The Smart Agricultural Marketplace Platform removes the information, trust, and payment gaps that exist between farmers and buyers. By applying modular design, security, scalability, and an Agile Scrum approach with DevOps, the system can be delivered in small, valuable increments, adapt to changing market needs, and provide a transparent, efficient, and fair agricultural marketplace for all participants.


AgriMarket
RP
Ramesh Patil
Verified farmer
Dashboard
My Products
Add Product
Orders
Delivery Tracking
Analytics
Feedback
Total products
18
+2 this month
Active orders
12
4 ready to pack
Completed orders
148
+18% this month
Total sales
₹2,84,600
+12.4% this month
Monthly sales
Revenue performance over the last 6 months
Last 6 months
₹2,84,600
↗ 12.4% vs last period
₹68k
Jan
Feb
Mar
Apr
May
Jun
Product availability
Current stock overview
View all
Fresh Tomatoes
480 kg available
In stock
Organic Potatoes
650 kg available
In stock
Red Onions
320 kg available
Low stock
Sona Masoori Rice
1200 kg available
In stock